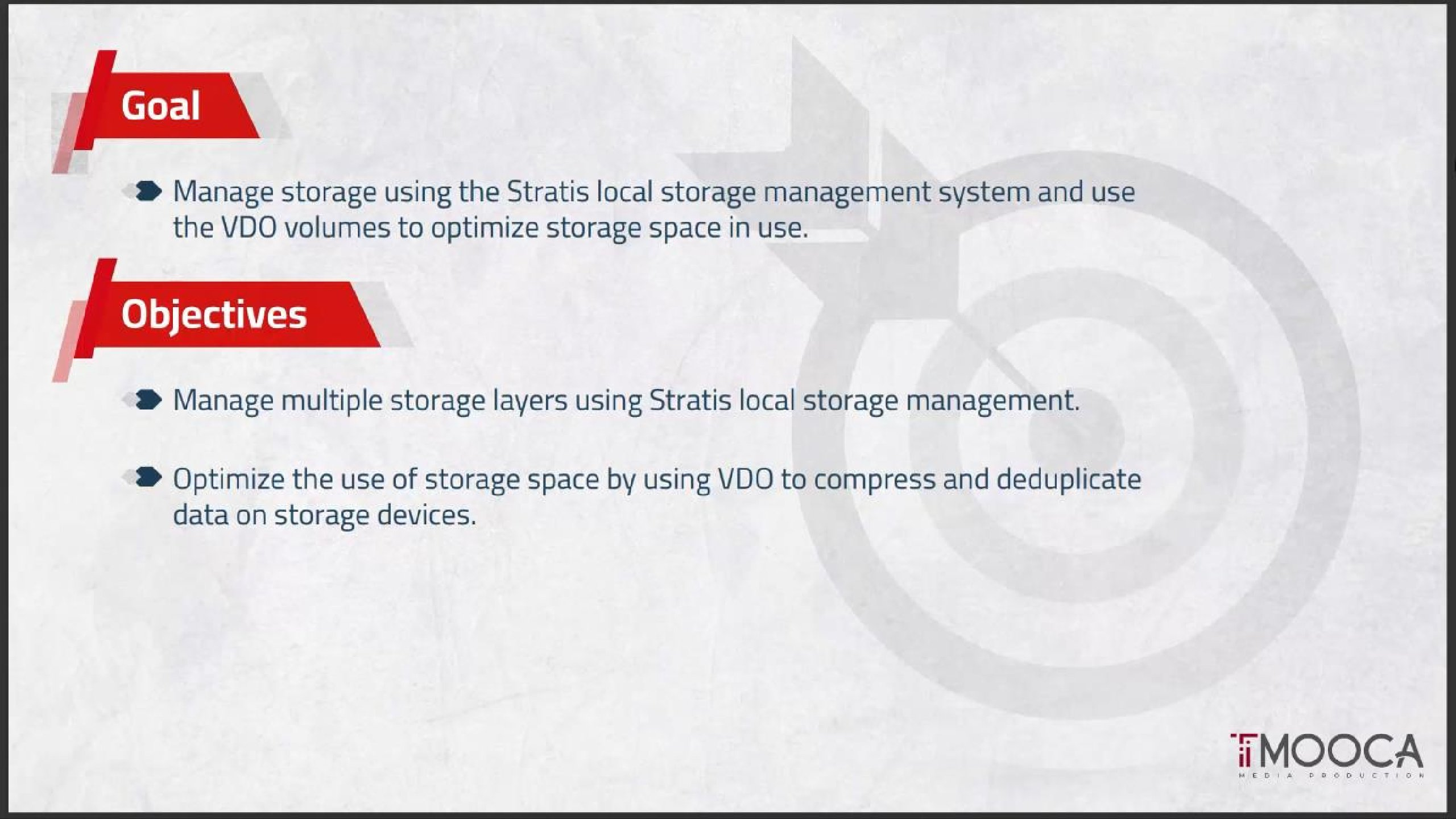


RH134

Implementing Advanced Storage Features



Red Hat | Enterprise Linux



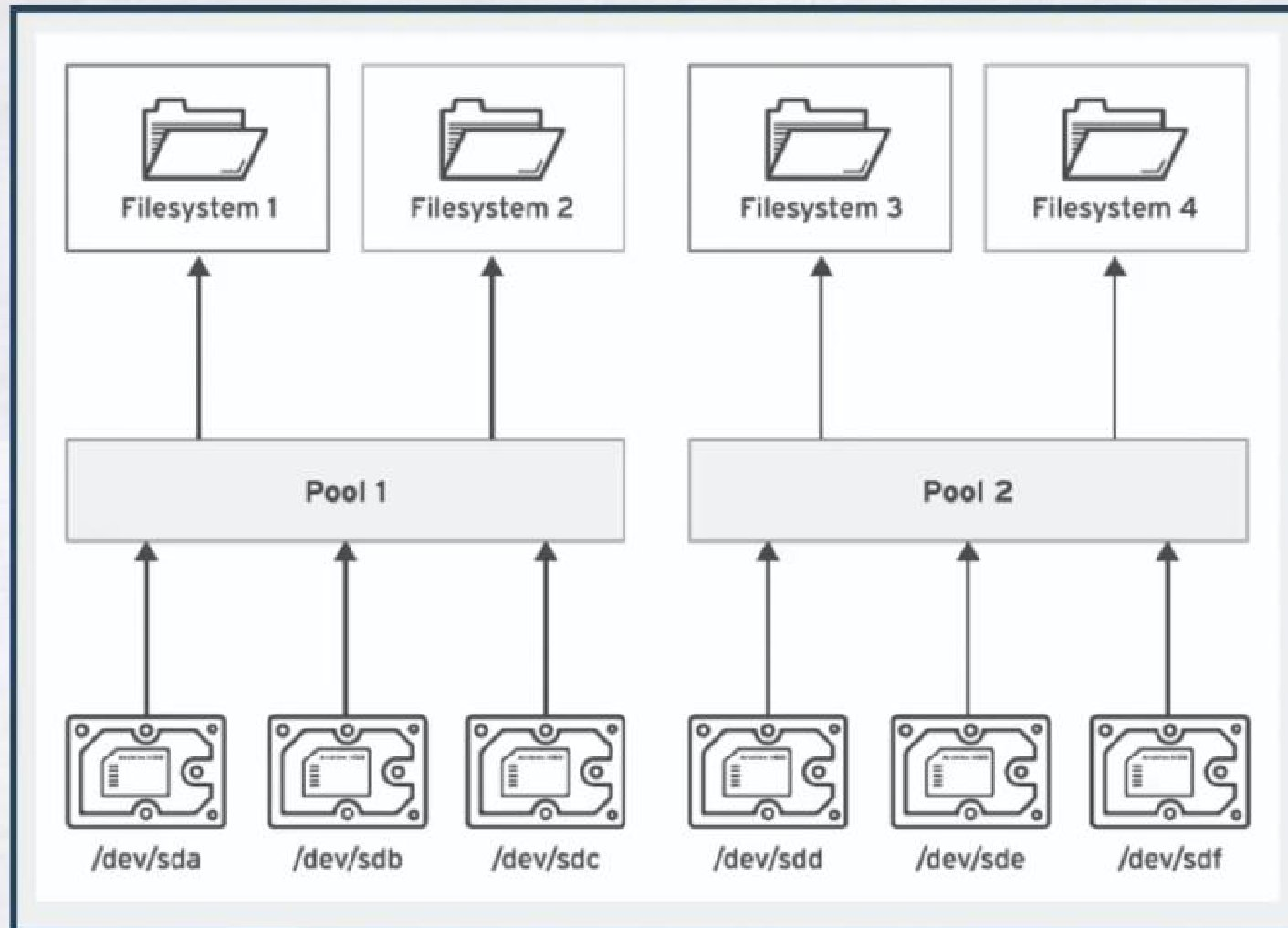
Goal

- Manage storage using the Stratis local storage management system and use the VDO volumes to optimize storage space in use.

Objectives

- Manage multiple storage layers using Stratis local storage management.
- Optimize the use of storage space by using VDO to compress and deduplicate data on storage devices.

Managing Storage with Stratis



○ **Stratis** هو عبارة عن أداة حديثة لإدارة التخزين (**Storage Management Solution**) تستخدم بسهولة إدارة الـ **Storage** عن طريق تكوين **Pool** من مجموعة مختلفة من الـ **Block Devices** (مثل الأقراص أو الـ **Partitions** مثل `dev/sda/` و `dev/sdb/` وغيرها كما يظهر في الصورة).

○ الـ **Pool** يعمل كـ **Backend Storage** أو الأساس الذي يتم من خلاله تخصيص وإدارة المساحات. وبمجرد إنشاء الـ **Pool**، يمكنك إنشاء مجموعة من الـ **File Systems** (مثل **1, 2, 3, 4**) داخله، كما يمكنك إنشاء أكثر من **Pool** على نفس النظام باستخدام **Block Devices** مختلفة.

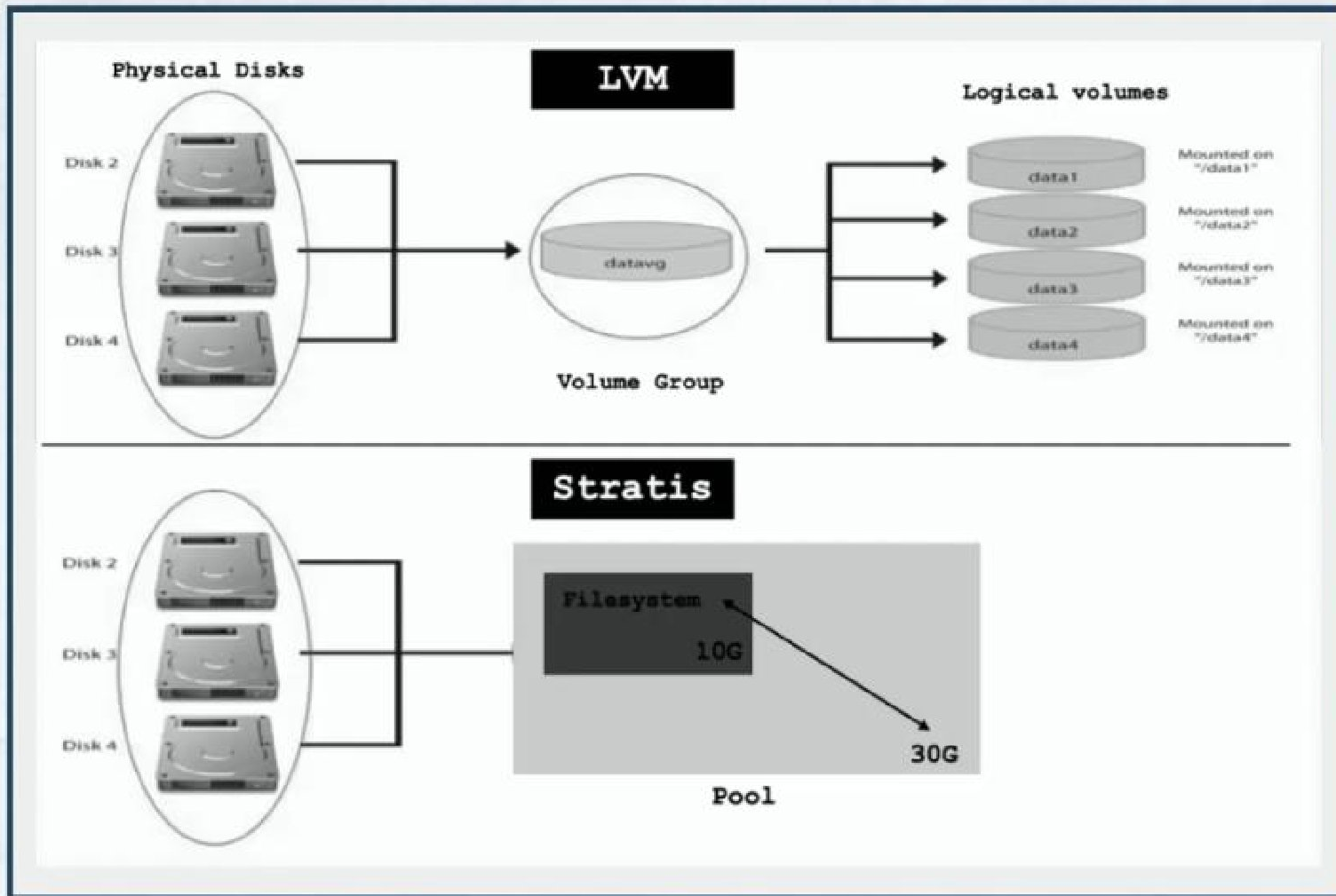
:Key Features of Stratis

○ **Thin Provisioning**: عند عمل **Create** لأي **File System**، يقوم **Stratis** بعمل **Dynamic Allocation** للمساحة بدلاً من تخصيص **Fixed Size**. بمعنى أن الـ **File System** يكبر تلقائياً (**Grows Up As Needed**) كلما أضفت **More Data** عليه، على عكس نظام الـ **LVM** التقليدي الذي كان يفرض حجماً ثابتاً منذ البداية.

○ **Automatic XFS Formatting**: بمجرد أن تقوم بـ **Create** للـ **File System** باستخدام **Stratis**، فإنه يقوم تلقائياً بـ **Format** كـ **XFS File System** دون أن تحتاج لعمل هذه الخطوة يدوياً أو إدارة تفاصيل نظام الملفات بنفسك.

○ **Snapshot Support**: يدعم **Stratis** ميزة الـ **Snapshot** لأخذ لقطات لحظية من الـ **File System**، والتي تُستخدم بفاعلية في عمليات الـ **Backup and Recovery Operations**، أو لاسترجاع الملفات المحذوفة بالخطأ (**Removed by Mistake**)، وحتى في حالة حدوث **File System Corruption** يمكنك استعادة البيانات بسهولة من خلال الـ **Snapshot**.

LVM vs. Stratis



○ LVM vs. Stratis Overview

- كلاهما يعتبران **Storage Management Solutions** يتم استخدامهما لإدارة التخزين، ويقوم الـ **System Administrator** باختيار النوع المناسب بناءً على الـ **Use Case** المطلوبة.

○ Stratis (Ease of Use & Simplicity)

- يتميز الـ **Stratis** بأنه أسهل بكثير في الـ **Setup** والتعامل مقارنة بـ **LVM**.
- الهيكلية بسيطة جداً: تأخذ الـ **Physical Devices** (الأقراص الفيزيائية)، تنشئ منها **Stratis Pool**، ومن ثم تقوم بإنشاء الـ **File System** مباشرة دون تعقيدات.

○ LVM (Complexity & Advanced Features)

- الـ **LVM** يكون الـ **Setup** الخاص به **More Complex** لأنه يحتوي على طبقات وخطوات متعددة (Physical Volumes -> Volume Groups -> Logical Volumes).
- في المقابل، يوفر الـ **LVM** ميزات متقدمة (**More Features**) وتحكماً أكبر قد لا تكون متوفرة بنفس الطريقة في الـ **Stratis**.

○ Shared Features (الميزات المشتركة):

- يتشابه كل من **LVM** و **Stratis** في دعمهم المشترك لميزة الـ **Snapshot** لأخذ لقطات لحظية للبيانات.

Working with Stratis Storage

- To manage file systems with the Stratis storage management solution, install the **stratis-cli** and **stratisd** packages.
- The stratis-cli package provides the stratis command, which sends reconfiguration requests to the stratisd system daemon.
- The stratisd package provides the stratisd service, which handles reconfiguration requests and manages and monitors block devices, pools, and file systems that Stratis uses.

Installing and Enabling Stratis

- To use Stratis, make sure that the software is installed and the **stratisd** service is running.
 - Install stratis-cli and stratisd using the **yum install** command.

```
[root@host ~]# yum install stratis-cli stratisd
```

- Activate the **stratisd** service using the **systemctl** command.

```
[root@host ~]# systemctl enable --now stratisd
```

o للتعامل مع ملفات النظام باستخدام **Stratis storage management solution**. يجب تثبيت حزمتي **stratis-cli** و **stratisd**.

o حزمة **stratis-cli** توفر أمر **stratis** الذي يرسل طلبات إعادة التكوين (reconfiguration requests) إلى الـ **stratisd** **system daemon**.

o حزمة **stratisd** توفر خدمة **stratisd** **service** المسؤولة عن التعامل مع طلبات إعادة التكوين وإدارة ومراقبة الـ **block devices, pools**, و **file systems**.

:Installing and Enabling Stratis

o لكي تستخدم **Stratis**, يجب التأكد من تثبيت البرمجيات وأن خدمة **stratisd** تعمل بنجاح.

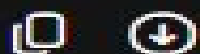
o تثبيت حزمتي **stratis-cli** و **stratisd** باستخدام أمر **yum install**:



Bash

```
yum install stratis-cli stratisd
```

o تفعيل وتشغيل خدمة **stratisd** باستخدام أمر **systemctl**:



Bash

```
systemctl enable --now stratisd
```

Assembling Block Storage into Stratis Pools

- The following are common management operations performed using the Stratis storage management solution.
 - Create pools of one or more block devices using the **stratis pool create** command.

```
[root@host ~]# stratis pool create pool1 /dev/vdb
```

- Use the **stratis pool list** command to view the block devices of a pool.

```
[root@host ~]# stratis pool list
```


Assembling Block Storage into Stratis Pools

- The following are common management operations performed using the Stratis storage management solution.
 - Use the **stratis pool add-data** command to add additional block devices to a pool.

```
[root@host ~]# stratis pool add-data pool1 /dev/vdc
```

- Use the **stratis blockdev list** command to view the block devices of a pool.

```
[root@host ~]# stratis blockdev list pool1
```

Managing Stratis File Systems

- Use the **stratis filesystem create** command to create a file system from a pool.

```
[root@host ~]# stratis filesystem create pool1 fs1
```

- Use the **stratis filesystem list** command to view the list of available file systems.

```
[root@host ~]# stratis filesystem list
```

- You can create a snapshot of a Stratis-managed file system with the **stratis filesystem snapshot** command. Snapshots are independent of the source file systems.

```
[root@host ~]# stratis filesystem snapshot pool1 fs1 snapshot1
```


Persistently Mounting Stratis File Systems

- To ensure that the Stratis file systems are persistently mounted, edit **/etc/fstab** and specify the details of the file system. The following command displays the UUID of the file system that you should use in **/etc/fstab** to identify the file system.

```
[root@host ~]# lsblk --output=UUID /stratis/pool1/fs1
```

- The following is an example entry in the **/etc/fstab** file to persistently mount a Stratis file system. This example entry is a single long line in the file.

```
UUID=31b9363b-add8-4b46-a4bf-c199cd478c55 /dir1 xfs defaults,xsystemd.requires=stratisd.service 0 0
```

- The **x-systemd.requires=stratisd.service** mount option delays mounting the file system until after **systemd** starts the **stratisd.service** during the boot process.

1. الشرح الخاص بالاسلايد الأول (Assembling Block Storage into Stratis Pools):

○ إنشاء ال Pool:

استخدام أمر `stratis pool create pool1 /dev/vdb` لإنشاء Stratis Pool جديد باسم `pool1` مستخدماً مساحة القرص الخام أو ال Block Device المسمى `/dev/vdb`.

○ عرض ال Pools:

استخدام أمر `stratis pool list` لاستعراض وعرض كافة ال Pools المتاحة على النظام.

2. الشرح الخاص بالاسلايد الثاني (إدارة وتوسعة ال Pools وعرض الأقراص):

○ إضافة مساحة لل Pool:

استخدام أمر `stratis pool add-data pool1 /dev/vdc` لإضافة قرص أو Block Device جديد (`/dev/vdc/`) إلى ال Pool الموجود بالفعل (`pool1`) لتوسيع مساحته الإجمالية.

○ عرض الأقراص المكونة لل Pool:

أمر `stratis blockdev list pool1` يكافئ تماماً أوامر مثل `pvs` أو `pvdiskdisplay` في عالم ال LVM، حيث يتيح لك رؤية الأقراص الفيزيائية (Physical Block Devices) الفعلية التي تبني وتدعم هذا ال Pool.

3. الشرح الخاص بالاسلايد الثالث (Managing Stratis File Systems):

○ إنشاء ال File System:

استخدام أمر `stratis filesystem create pool1 fs1` لإنشاء File System جديد باسم `fs1` داخل ال Pool `pool1`. هنا أنت فقط تحدد اسم ال Pool واسم ال File System المطلوب إنشاؤه.

○ التنسيق التلقائي (Automatic XFS Formatting):

بمجرد تنفيذ أمر الإنشاء، يقوم Stratis تلقائياً بعمل Format لل File System بنظام ملفات XFS بالنيابة عنك ودون الحاجة لتنفيذ أوامر التنسيق اليدوية المعتادة.

○ عرض واستخراج Snapshots:

○ لعرض ال File Systems المتاحة: `stratis filesystem list`

○ لأخذ لقطة (Snapshot): `stratis filesystem snapshot pool1 fs1 snapshot1`: لإنشاء نسخة مستقلة تماماً عن ال File System الأصلي لحماية البيانات واستعادتها عند الحاجة.

4. الشرح الخاص بالاسلايد الرابع والأخير (Persistently Mounting Stratis File Systems):

○ التثبيت الدائم في النظام:

لكي يظل ال File System مربوطاً (Mounted) بانتظام بعد إعادة تشغيل الجهاز، نحتاج لتعديله وإضافته في ملف `etc/fstab/`.

○ الحصول على ال UUID:

نستخدم أمر `lsblk --output=UUID /stratis/pool1/fs1` لاستخراج ال UUID الخاص بال File System لنستضفه في ملف ال `fstab`.

○ شكل السطر في `etc/fstab/`:

```
UUID=31b9363b-add8-4b46-a4bf-c199cd478c55 /dir1 xfs defaults,x-  
systemd.requires=stratisd.service 0 0
```

○ أهمية إضافة خيار `x-systemd.requires=stratisd.service`:

تمت إضافة هذا الجزء خصيصاً لتأخير عملية ال Mount لهذا ال File System لحين التأكد من أن نظام ال systemd قد قام بتشغيل خدمة `stratisd.service` أولاً أثناء عملية الإقلاع (Boot). لو لم يتم وضع هذا الخيار، سيحاول النظام عمل Mount قبل إقلاع خدمة Stratis نفسها، مما يفشل في قراءة القرص ويدخلك في مشاكل وضع ال Emergency Mode.

- o `dnf search stratis`
- o `dnf install stratis-cli stratisd`
- o `systemctl enable --now stratisd`
- o `systemctl status stratisd`
- o `lsblk -p`
- o `stratis pool create stratispool1 /dev/nvme0n2`
- o `stratis pool list`
- o `stratis pool add-data stratispool1 /dev/nvme0n3`
- o `stratis pool list`
- o `stratis blockdev list`
- o `stratis blockdev list stratispool1`
- o (ملحوظة: لو عندك أكثر من *Pool*، بتدي له اسم ال *Pool* وتحدده في الأمر)
- o `stratis filesystem create stratispool1 stratis-filesystem1`
- o `stratis filesystem list`
- o (ملحوظة: ال *Total* اللي هتلاقيه *1 TiB* تقريباً تجاهله دي أصلاً، هو بس بيشير لأن المساحة ديناميكية وليست *Fixed Size*)
- o `stratis filesystem create stratispool1 stratis-filesystem2`
- o (بعد ما ينفذ الإثنين فايل سيستم: كأنك عملت بال *LVM* بس وخلاص)

mkdir /stratis-filesystem1 ○

mkdir /stratis-filesystem2 ○

mount /dev/stratis/stratispool1/stratis-filesystem1 /stratis-filesystem1 ○

df -h ○

○ (ملحوظة: تجاهل مساحة الـ 1 تيرابايت التي تظهر في أمر `df -h`)

○ stratis pool أو stratis filesystem (لمعرفة الـ *Actual Utilization* الحقيقية)

mount /dev/stratis/stratispool1/stratis-filesystem2 /stratis-filesystem2 ○

df -h ○

lsblk --output=UUID /dev/stratis/stratispool1/stratis-filesystem1 ○

lsblk --output=UUID /dev/stratis/stratispool1/stratis-filesystem2 ○

○ stratis filesystem list

○ blkid

○ (ملحوظة: دي الطرق المختلفة اللي تقدر تجيب بيها الـ *UUID* الخاص بالـ *Filesystem*)

echo "UUID=c82e3b74-44f1-8a4a-ccd0767cb59b /stratis-filesystem1 xfs defaults,x-
systemd.requires=stratisd.service 0 0" >>/etc/fstab ○

echo "UUID=b10f8fd7-22d5-4d38-9b49-1a3f743f9aee /stratis-filesystem2 xfs defaults,x-
systemd.requires=stratisd.service 0 0" >>/etc/fstab ○

○ (ملحوظة: تم عمل *Append* مباشرة للـ *File* باستخدام << بدل فتحه والتعديل عليه يدوياً)

cat /etc/fstab ○

systemctl daemon-reload ○

mount -a ○

systemctl reboot ○

○ (ملحوظة: اضغط على حرف **R** لإعادة تشغيل الـ *Session* بعد انقطاع الاتصال نتيجة إعادة الإقلاع)

df -h ○

○ (ملحوظة: السيسـتم أول ما عمل *Reboot*، قام بتشغيل خدمة *Stratis* (*stratisd.service*) تلقائياً في الخلفية بفضل أمر التبعية *x-systemd.requires=stratisd.service* الذي أضفناه مسبقاً في ملف *etc/fstab/*، وبناءً على ذلك تم عمل *Mount* للـ *File Systems* تلقائياً وينجح دون أي تدخل يدوي)

stratis filesystem list ○

○ (ملحوظة: ستلاحظ أن الـ *Used* أو الـ *Actual Utilization* الخاص بالـ *Filesystem* يظهر بمساحة صغيرة جداً تقدر بحوالي 546 ميجابايت أو أقل قبل إضافة أي ملفات)

echo "Hello World!" >file1 ○

|dd if=/dev/urandom of=file2 bs=1M count=1024 ○

* du -sh ○

stratis filesystem list ○

○ (ملحوظة: بعد إضافة الملفات، سيتغير حجم الاستخدام الفعلي (*Used*) في القائمة ليطابق المساحة الجديدة بناءً على التخصيص التلقائي والديناميكي لـ *Stratis*)

```
stratis filesystem list
```

```
stratis filesystem snapshot stratispool1 stratis-filesystem1 stratis-filesystem1-snap
```

- شرح الأمر: أخذ نسخة مطابقة (snapshot) للحالة الحالية للـ `filesystem` الأصلي وحفظها كنسخة جديدة داخل الـ `pool` بالاعتماد على التخصيص الديناميكي دون استهلاك مساحة كاملة فوراً.

```
mkdir /stratis-filesystem1-snap
```

```
mount /dev/stratis/stratispool1/stratis-filesystem1-snap /stratis-filesystem1-snap
```

- توضيح الـ `mount` وإجابة تفصيلية حول المساحة:

- عندما تقوم بعمل `mount` للـ `snapshot`، فهو يعامل كـ `filesystem` مستقل بذاته تماماً، يمكن التعديل بداخله أو قراءته بمعزل تام عن الأصل.
- إذا كانت مساحة الـ `filesystem` الأصلي أثناء عمل الـ `snapshot` تساوي 1 جيجا، فهو لا يحجز المساحة كاملة فوراً لمجرد إنشائه أو عمل `mount` لها.
- يعتمد في بدايته على مفهوم **Copy-on-Write (CoW)**، يعني يعمل مشاركة نفس البلوكات والبيانات الأصلية دون تكرارها، حتى لو متكررة ما يكرر هاش.
- تبدأ مساحته الفعلية من الصفر، ولا يستهلك سوى مساحة ضئيلة جداً بالـ `metadata` الخاصة به.
- متى تزداد ومن أين تأتي؟ تزداد المساحة فعلياً عندما تقوم بتعديل ملف قديم أو إضافة ملفات جديدة بداخله؛ حيث يتم نسخ البيانات الجديدة أو المعدلة، ويتم اقتطاع هذه المساحة مباشرة من الـ `pool size`.

```
df -h o
cd /stratis-filesystem1-snap o
ls o
cat file1 o
```

ملحوظة: هتلاحظ إن الملف اللي كنا حاسبينه موجود أهو.

خطوات إدارة أنظمة ملفات ومجموعات تخزين Stratis تشمل عرض الحالة، إلغاء الربط (Umount)، أخذ النسخ الاحتياطية (Snapshots)، حذف أنظمة الملفات وال Pools، والتحقق من الأقراص.

```
stratis filesystem list o
umount /stratis-filesystem1 o
stratis filesystem list o
stratis filesystem snapshot stratispool1 stratis-filesystem1 stratis-filesystem1-snap o
```

شرح الأمر: يُستخدم هذا الأمر لإنشاء نسخة احتياطية لحظية (Snapshot) من ال Filesystem الأصلي (stratis-filesystem1) داخل ال Pool (stratispool1) وتسميتها stratis-filesystem1-snap ، بحيث يتم حفظ حالة الملفات في تلك اللحظة الزمنية بدقة دون استهلاك مساحة كاملة فوراً.

```
stratis filesystem list o
mount /dev/stratis/stratispool1/stratis-filesystem1 /stratis-filesystem1 o
cd /stratis-filesystem1 o
ls -l o
```

ظهور الملفات ومحتوياتها بداخل ال Filesystem.

`stratis filesystem list` ○

`umount /stratis-filesystem1` ○

`umount /stratis-filesystem2` ○

`umount /stratis-filesystem1-snap` ○

`stratis filesystem destroy stratispool1 stratis-filesystem1` ○

○ **الهدف من عملية ال destroy لأنظمة الملفات:** نلجأ لحذف ال filesystems عندما لا نكون بحاجة إليها، أو لتنظيف المساحة، أو إعادة هيكلة التخزين داخل ال Pool.

○ **شرح صيغة الأمر:** يأخذ الأمر اسم ال Pool متبوعاً باسم ال Filesystem المراد حذفه (مثل `stratispool1 stratis-filesystem1`) لتحديد المساحة والفايل سيستم بدقة وحذفه نهائياً.

`stratis filesystem destroy stratispool1 stratis-filesystem2` ○

`stratis filesystem destroy stratispool1 stratis-filesystem1-snap` ○

`stratis filesystem list` ○

`stratis pool list` ○

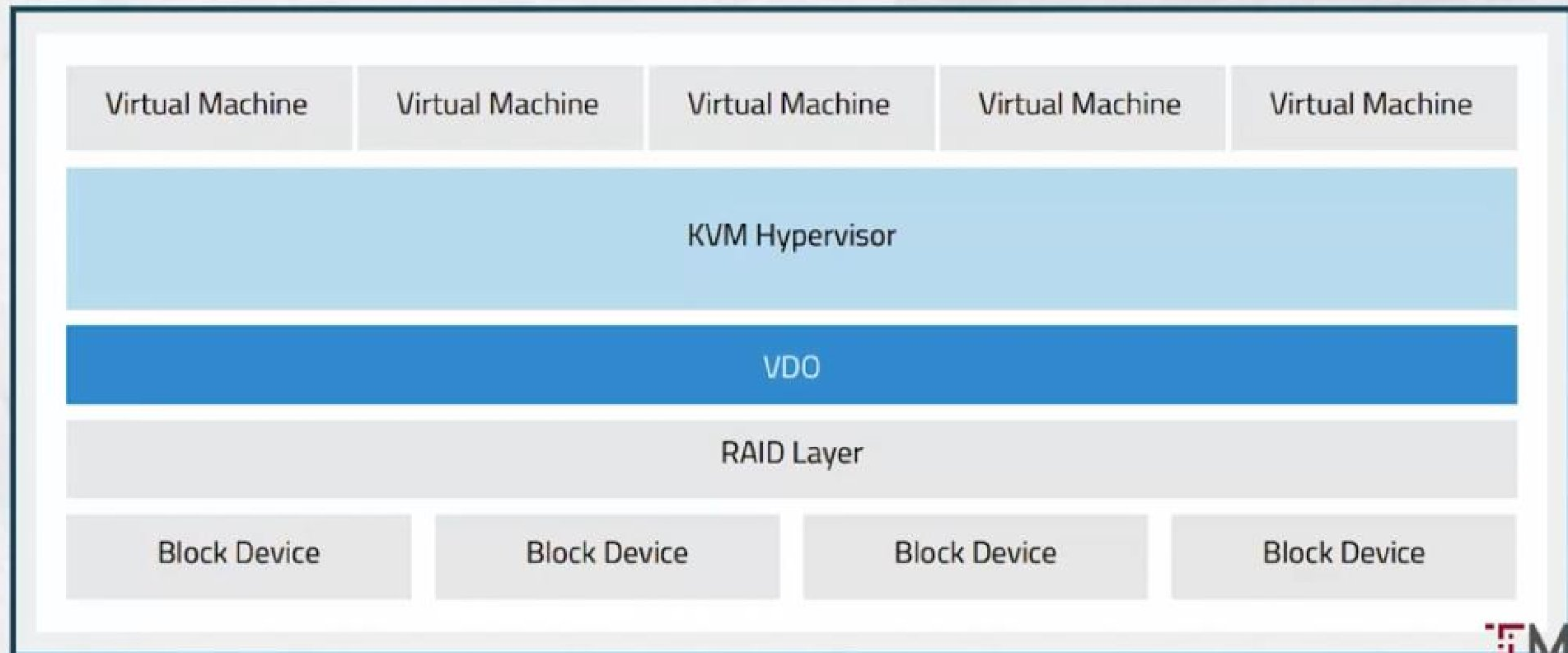
`stratis pool destroy stratispool1` ○

`stratis pool list` ○

`lsblk -p` ○

Compressing and Deduplicating Storage with VDO

- The Virtual Data Optimizer (VDO) is a storage optimization technology developed by Red Hat. It is designed to reduce storage costs and improve efficiency by using data deduplication and compression techniques. VDO operates at the block level and is primarily used for reducing the amount of storage space required for storing data.



تقنية (VDO) Virtual Data Optimizer المطورة من قبل Red Hat تهدف أساساً إلى تحسين كفاءة التخزين وخفض التكاليف من خلال معالجة البيانات مركزياً.

إليك التفاصيل مع إبراز الكلمات والمصطلحات الرئيسية باللغة الإنجليزية:

○ **التعريف الأساسي (Core Definition):**

○ هي تقنية Storage Optimization (تحسين التخزين) طورتها Red Hat لتقليل Storage Costs (تكاليف التخزين) وزيادة الكفاءة عبر تقنيات Data Deduplication (إزالة التكرار) و Compression (الضغط).

○ تعمل هذه التقنية على مستوى الـ Block Level لتقليل المساحة الفعلية المطلوبة لحفظ البيانات.

○ **هيكلية الطبقات (Architecture Layers):**

- Virtual Machines (VMs) / VMware: الآلات الافتراضية التي تعمل في الطبقة العليا.
- KVM Hypervisor: طبقة المحاكاة الافتراضية التي تدير النظام.
- VDO (Virtual Data Optimizer): الطبقة الذكية والوسيط للضغط وإزالة التكرار.
- RAID Layer / Storage Pool: طبقة تجميع الأقراص والسعة الخام (مثل الـ Volume Group أو الـ Stratis Pool).
- Block Devices: وحدات التخزين الفيزيائية الأساسية.

○ **الميزة المركزية والنتيجة (Centralized Advantage & Result):**

- بدلاً من تطبيق عمليات الضغط وإزالة التكرار لكل آلة افتراضية (VM / VMware) على حدة وبشكل منفصل، يتم تطبيق الـ VDO مركزياً على مستوى الـ Storage Pool بالكامل.
- **النتيجة:** أي بيانات تُكتب من أي VM تمر عبر هذه الطبقة الموحدة، فيتم ضغطها وإزالة التكرار منها مرة واحدة على مستوى الـ Pool، مما يوفر مساحة ضخمة ويهيكل التخزين بكفاءة عالية جداً دون الحاجة لإدارتها لكل جهاز على حدة.

➤ **VDO applies three phases to data in the following order to reduce the footprint on storage devices:**

1. Zero Block Elimination:

- In this phase, VDO identifies and eliminates blocks of data that consist entirely of zeros. These blocks are often found in sparse files or during the initial stages of data writes. By eliminating zero blocks, VDO can quickly reclaim unused storage space.

2. Data Compression:

- After zero block elimination, VDO employs data compression techniques to reduce the size of the remaining non-zero data blocks. Data compression algorithms are applied to individual blocks before they are stored on the storage device. This reduces the amount of physical space required to store the data.

3. Deduplication:

- In the deduplication phase, VDO identifies duplicate data blocks and stores only one instance of each unique block. This process is crucial for eliminating redundant copies of data that may be present across different files or versions. Deduplication contributes significantly to reducing the overall storage footprint.

تمر تقنية الـ VDO بثلاث مراحل أساسية وبالترتيب لمعالجة البيانات وتقليل المساحة التي تشغلها:

1. Zero Block Elimination (حذف البلوكات الصفرية):

- الفكرة: يقوم النظام (system) بالبحث عن كتل البيانات المكونة بالكامل من أصفار (blocks of data that consist entirely of zeros) والتخلص منها نهائياً.
- أين توجد؟ تظهر غالباً في الملفات المتفرقة (sparse files) أو أثناء المراحل الأولى لكتابة البيانات (initial stages of data writes).
- الهدف: استعادة مساحة التخزين غير المستخدمة بسرعة (reclaim unused storage space).

2. Data Compression (ضغط البيانات):

- الفكرة: بعد التخلص من البلوكات الصفرية (zero block elimination)، يعتمد النظام على خوارزميات الضغط (data compression techniques) لتقليص حجم البلوكات المتبقية التي تحتوي على بيانات فعلية غير صفرية (non-zero data blocks).
- كيف تعمل؟ تُطبق خوارزميات الضغط على كل بلوك بشكل فردي (individual blocks) قبل تخزينه فعلياً على أجهزة التخزين (storage device).
- الهدف: تقليل المساحة الفيزيائية المطلوبة لحفظ البيانات (physical space required to store the data).

3. Deduplication (إزالة التكرار):

- الفكرة: في هذه المرحلة، يحدد النظام كتل البيانات المكررة (duplicate data blocks) ويقيم بتخزين نسخة واحدة فقط لكل بلوك فريد (store only one instance of each unique block).
- كيف تعمل؟ تتخلص من النسخ الزائدة والمتطابقة للبيانات (redundant copies of data) التي قد تكون موجودة عبر الملفات أو الإصدارات المختلفة (different files or versions).
- الهدف: تقليص إجمالي المساحة التخزينية المستهلكة بشكل جذري (reducing the overall storage footprint).

Virtual Data Optimizer (VDO)

➤ Enabling VDO

- Install the **vdo** and **kmod-kvdo** packages to enable VDO in the system.

```
[root@host ~]# yum install vdo kmod-kvdo
```

➤ Creating a VDO Volume

- To create a VDO volume, run the **vdo create** command.

```
[root@host ~]# vdo create --name=vdo1 --device=/dev/vdd --vdoLogicalSize=50G
```

➤ Analyzing a VDO Volume

- To analyze a VDO volume, run the **vdo status** command.

```
[root@host ~]# vdo status --name=vdo1
```

1. Enabling VDO (تفعيل الحزم):

- لتفعيل الخدمة في النظام، يجب أولاً تثبيت حزم الـ VDO والتعريفات الخاصة بها عبر الأمر التالي:

```
yum install vdo kmod-kvdo
```

2. Creating a VDO Volume (إنشاء وحدة تخزين VDO):

- بعد تثبيت الحزم، يمكنك التعامل مع الـ partition المخصص وكأنه مساحة تخزين عادية.

```
vdo create --name=vdo1 --device=/dev/vdd --vdoLogicalSize=50G
```

- شرح الأمر: يقوم هذا الأمر بإنشاء حجم تخزيني وهمي أو افتراضي (Logical Size) بقيمة 50 جيجا على القرص الفيزيائي (/dev/vdd) وتسميته `vdo1`.

3. Analyzing a VDO Volume (فحص واستعراض الحالة):

- لاستعراض تفاصيل دقيقة ومعلومات متقدمة حول الـ VDO volume بعد إنشائه، تستخدم أمر الحالة:

```
vdo status --name=vdo1
```

ملحوظة إدارية هامة جداً (Red Hat 9 وما فوق - LVM Integration):

- بدءاً من إصدار Red Hat 9 والأحدث، أصبحت أوامر الـ `vdo standalone` القديمة غير مستخدمة (deprecated / no longer of use).

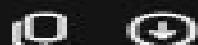
- تم دمج ميزة الـ VDO بشكل كامل وصريحة داخل نظام الـ LVM (Logical Volume Manager).

- لكي تطبق هذه الميزة حالياً، لم تعد بحاجة لأوامر الـ `vdo create` المنفصلة، بل عند قيامك بإنشاء `logical volume` جديد، كل ما عليك فعله هو إضافة `--type vdo` ضمن معلمات الأمر.

- بمجرد تفعيلها عبر الـ LVM، سيتم تطبيق المراحل الثلاث تلقائياً على أي بيانات يتم تخزينها، وهي: `Data Compression`، `Zero Block Elimination`، و `Deduplication`.

لتطبيق الـ VDO على مستوى الـ Storage Pool بالكامل أو الـ Volume Group بحيث تسري المعالجة تلقائياً على أي بيانات دون الحاجة لتطبيقها على كل بارتيشن على حدة، يتم عكس الهرم ووضع طبقة الـ VDO في القاع (تحت الـ Pool وليست فوقها).

1. جعل الـ VDO هو الأساس (في قاع الهرم):
أخذ قرص خام (Raw Disk) وليكن `dev/vdd/` وإنشاء مساحة VDO عليه مباشرة لتكون القاعدة الأساسية للتخزين:



Bash

```
vdo create --name=vdo1 --device=/dev/vdd --vdoLogicalSize=100G
```

2. بناء الـ Physical Volume والـ Volume Group فوق الـ VDO:
التعامل مع جهاز الـ VDO الناتج وكأنه قرص حقيقي، حيث تحوله إلى Physical Volume ثم تبني مجموعة التخزين (VG) فوقه، ليصبح أي Logical Volume تنشئه داخل هذه الـ VG محمي ومُحسّن بتلقائية:



Bash

```
pvcreate /dev/mapper/vdo1  
vgcreate vdo_vg /dev/mapper/vdo1  
lvcreate -n vdo_lv -l 100%FREE vdo_vg
```

ملحوظة هامة:

بدءاً من إصدار Red Hat 9 وأحدث، لن تتمكن من تطبيق هذه الطريقة الكلاسيكية لأن أمر الـ `vdo create` المستقل أصبح ملغياً وغير مدعوم (`deprecated`). في RHEL 9، تم الاستغناء عن وضع الـ VDO في قاع الهرم، وأصبح تطبيقه يتم حصرياً بدمجه داخل الـ LVM بشكل مباشر أثناء إنشاء الـ Logical Volume عبر استخدام المعامل `--vdo-type`.

(تثبيت الحزم، إنشاء ال PV، ال VG، ال VDO، ال LV)

```
dnf install vdo kmod-kvdo
```

```
lsblk -p
```

```
pvccreate /dev/nvme2 /dev/nvme3
```

```
vgcreate vdo_vg /dev/nvme2 /dev/nvme3
```

```
vgs
```

```
lvcreate --type vdo -n vdo_lv -l +100%FREE vdo_vg
```

○ شرح المعامل `-l +100%FREE` : يعني تخصيص واستغلال كل المساحة الفارغة المتاحة (Free Space %100) بالكامل داخل ال Volume Group (`vdo_vg`) لصالح إنشاء ال Logical Volume الجديد دون الحاجة لتحديد حجم يدوي.

```
lvs
```

○ للطلاع على المزيد من الأمثلة وال Syntax الخاص بهذه العمليات وطريقة استخدام ال `lvchange` لتفعيل أو تعطيل ال Compression وال Deduplication، يمكنك استعراض ال manual الرسمي عبر الأمر:

```
man lvmvdo (أو man lvcreate)
```

(عرض خرائط وتعديل ال Compression وال Deduplication)

```
lvls -o lv_name,vdo_compression,vdo_deduplication
```

- لعرض أعمدة مخصصة (-o) تُظهر اسم ال **Logical Volume** (lv_name)، وحالة ضغط البيانات (vdo_compression)، وحالة إزالة التكرار (vdo_deduplication) .

```
lvchange --compression n /dev/vdo_vg/vdo_lv
```

- استخدام الحرف n (أي No) لتعطيل خاصية ضغط البيانات (Compression) .

```
lvchange --deduplication n /dev/vdo_vg/vdo_lv
```

- تعطيل خاصية إزالة التكرار (Deduplication) بوضع الحرف n .

```
lvchange --compression y /dev/vdo_vg/vdo_lv
```

- إعادة تفعيل (Enable) خاصية الضغط (Compression) بوضع الحرف y (أي Yes) .

```
lvchange --deduplication y /dev/vdo_vg/vdo_lv
```

- إعادة تفعيل خاصية إزالة التكرار (Deduplication) بوضع الحرف y .

```
lvls -o lv_name,vdo_compression,vdo_deduplication
```

- التحقق النهائي من حالة الخصائص للتأكد من تفعيلها أو تعطيلها بدقة.

```
mkfs -t xfs /dev/vdo_vg/vdo_lv
```

```
mkdir /vdo_lv
```

```
mount /dev/vdo_vg/vdo_lv /vdo_lv
```

```
df -h
```




```
cd /vdo_lv/  
vdostats  
vdostats --human-readable  
dd if=/dev/urandom of=urandom_file bs=1M count=1024  
du -sh urandom_file  
vdostats --human-readable  
cp urandom_file urandom_file1  
cp urandom_file urandom_file2  
cp urandom_file urandom_file3  
cp urandom_file urandom_file4  
du -sh urandom_file*
```

شرح مفهوم Space saving في أمر `vdostats` :

خانة `Space saving` توضح نسبة المساحة التي تم توفيرها بفضل تقنيات الـ VDO (مثل حذف البلوكات الصفيرية والضغط وإزالة التكرار). النسبة العالية (مثل 99%) تعني أن النظام وقر مساحة هائلة مقارنة بالحجم الحقيقي المكتوب قبل المعالجة، بينما انخفاضها مع ملفات `urandom` يعود لأن البيانات العشوائية يصعب ضغطها.

ملحوظة فنية حول زيادة المساحة (1GB):

عند استخدام الأمر `dd` لإنشاء ملف يعتمد على `/dev/urandom/` ، فإن البيانات الناتجة تكون عشوائية تماماً وغير قابلة للضغط أو الانضغاط لأنها لا تحتوي على أنماط متكررة أو بلوكات صفيرية، ولهذا السبب زادت المساحة المستخدمة (`Used`) في الـ `vdostats` بمقدار 1 جيجابايت فعلياً دون أي توفير في هذه الخطوة المحددة.

vdostats --human-readable o

cat /dev/zero > zero_file o

du -sh zero_file o

vdostats --human-readable o

cp zero_file zero_file1 o

vdostats --human-readable o

* rm -rf o

umount /vdo_lv o

lvremove /dev/vdo_vg/vdo_lv o

vgremove vdo_vg o

pvrremove /dev/nvme2 /dev/nvme3 o

تفسير عدم زيادة المساحة المستخدمة مع ملف الـ 2.4 جيجا وارتفاع الـ Space saving إلى 70%

السبب الرئيسي في عدم زيادة المساحة المستخدمة (Used Space) نهائياً على الرغم من إنشاء ملف بحجم 2.4 جيجابايت هو تقنية Zero Block Elimination (حذف البلوكات الصفيرية):

- عندما تستخدم المصدر `dev/zero/` لإنشاء الملف، فإن محتوى الملف بالكامل عبارة عن أصفار صفرية (zeros).

- تقوم طبقة الـ VDO بفحص البيانات القادمة وتكتشف أن هذا الملف لا يحتوي سوى على بلوكات صفيرية، فتقوم بالتخلص منها فوراً وعدم حجز أي مساحة حقيقية لها على القرص الفيزيائي.

- هذا هو السبب في أن حجم الملف الظاهري (`du -sh`) يظهر 2.4 جيجا، بينما مساحة التخزين الفعلية المستخدمة على الـ VDO لم تتغير، وارتفعت نسبة Space saving لتصل إلى 70% لأن النظام نجح في ضغط/إلغاء هذه البلوكات بالكامل وتوفير مساحتها.

توضيح عملية التراجع وحذف الـ VDO Logical Volume

تتم عملية الحذف العكسية (Rollback / Cleanup) بترتيب تنازلي يبدأ من فك الارتباط وصولاً إلى الأقراص الخام:

- لماذا تم استخدام `lvremove` لحذف الـ VDO؟ لأننا في إصدارات Red Hat 9+ أنشأنا الـ VDO مدمجاً مباشرة داخل الـ Logical Volume باستخدام المعامل `type vdo--`، ولم ننشئه كجهاز مستقل في قاع الهرم.

- بناءً على ذلك، فإن حذف الـ Logical Volume عبر أمر `lvremove /dev/vdo_vg/vdo_lv` يؤدي تلقائياً إلى تدمير وإزالة طبقة الـ VDO والـ Pool المرتبطة به المحتواة داخله.

- بعد ذلك يتم استكمال الحذف الهرمي طبيعياً عبر إلغاء تفعيل الـ Volume Group بـ `vgremove vdo_vg`، وأخيراً مسح الـ Physical Volumes بأمر `pvremove`.

إذا أردت توسيع مساحة الـ VDO الحالية بعد إضافة قرص جديد إلى الـ Volume Group، فلن تحتاج لإنشاء VDO جديد منفصل، بل تقوم بتوسيع الـ Pool الحالي (vpool0) ثم تكبير المساحة الظاهرية (vdo-lv).



Bash

```
vgextend vdo_vg /dev/nvmeXnY  
lvextend -L +3G /dev/vdo_vg/vpool0  
lvextend -L +3G /dev/vdo_vg/vdo-lv
```

② تحديث حجم نظام الملفات

Filesystem Resize

بعد توسيع المساحة الظاهرية، يجب إختيار نظام الملفات (XFS) بأن المساحة زادت ليراها أمر `df -h`:



Bash

```
xfs_growfs /vdo_mount
```

③ التحقق من المساحة الجديدة

Verification

تأكد من أن المساحة الإجمالية قد زادت بنجاح:



Bash

```
df -h
```


Summary

In this chapter, you learned:

- The Stratis storage management solution implements flexible file systems that grow dynamically with data.
- The Stratis storage management solution supports thin provisioning, snapshotting, and monitoring.
- The Virtual Data Optimizer (VDO) aims to reduce the cost of data storage.
- The Virtual Data Optimizer applies zero-block elimination, data deduplication, and data compression to optimize disk space efficiency.



RH134

Accessing Network-Attached Storage



Red Hat | Enterprise Linux

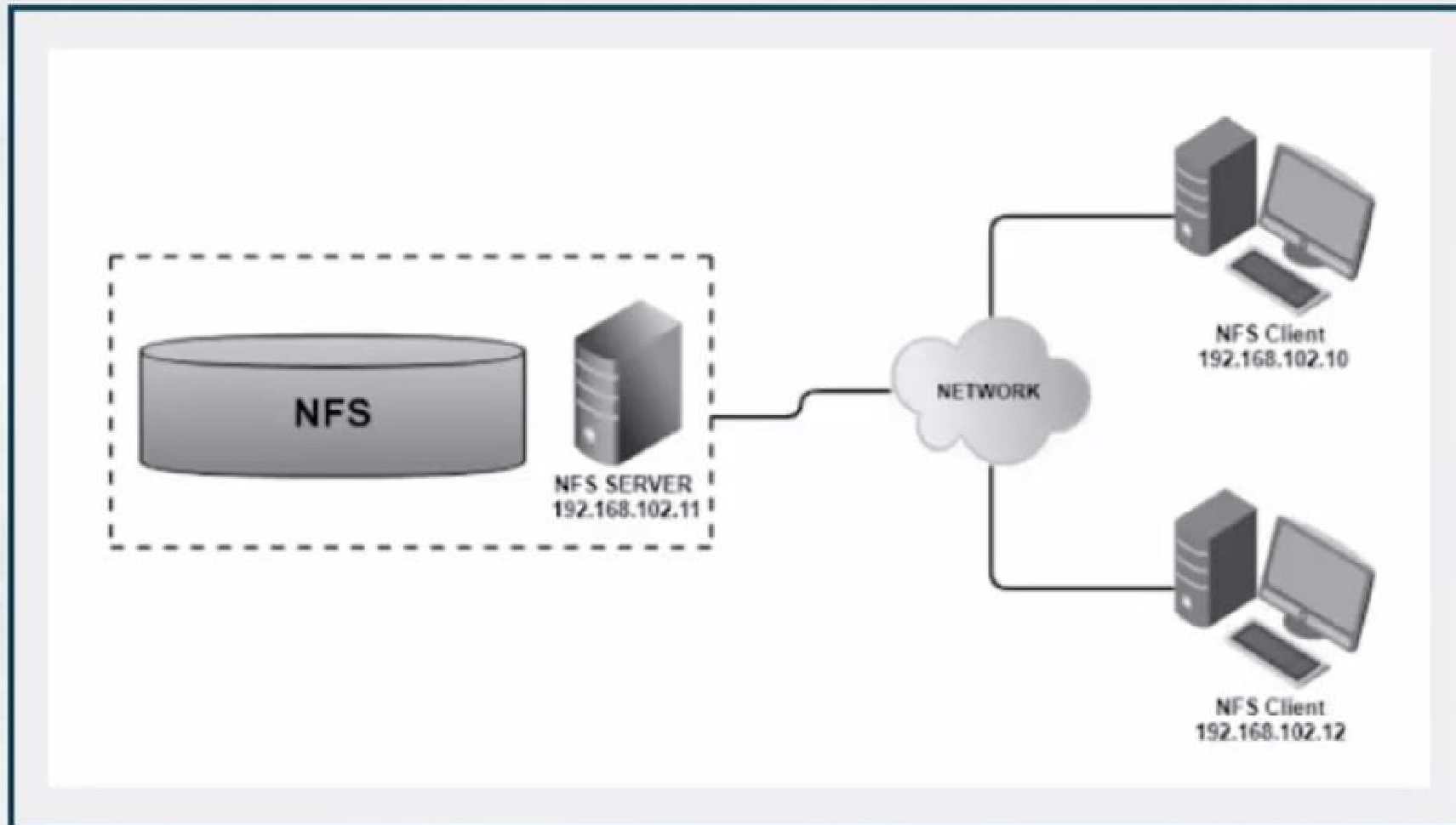
Goal

- Access network-attached storage using the NFS protocol.

Objectives

- Mount, use, and unmount an NFS export from the command line and at boot time.
- Configure the automounter with direct and indirect maps to automatically mount an NFS file system on demand, and unmount it when it is no longer in use.

Accessing Network Storage with Network File System (NFS)



- **NFS (Network File System):** هو بروتوكول شبكي يدعم مشاركة الملفات (Network File Sharing) بين أنظمة تشغيل Linux و Unix.
- **مشاركات أخرى (Linux & Windows):** إذا كنت تريد مشاركة الملفات بين أنظمة Linux و Windows، فهناك بروتوكولات أخرى مثل **SMB** (أو ما يُعرف بـ CIFS).

- يتوفر لبروتوكول NFS عدة إصدارات مثل v2 و v3 و v4.
- الإصدار الافتراضي في Red Hat 9 هو NFS 4.2.
- **الدعم القديم:** الإصدار NFS v3 لا يزال مدعماً في RHEL 9، مما يمنحك خيار الاختيار بين الإصدارين، لكن الإصدار 4 يمتلك مميزات متقدمة جداً مقارنة بـ v3.

- **الاعتماد على TCP:** يستخدم NFSv4 بروتوكول TCP في الاتصال حصرياً، عكس الـ old versions (الإصدارات القديمة) التي كانت تعتمد على الـ TCP والـ UDP معاً.
- **المنافذ (Ports):** الإصدارات القديمة لم يكن لها بورت ثابت، وكانت الـ services تعتمد على services أخرى مثل الـ RPC bind (التي كانت تعمل على بورت 111)، أما في الإصدارات الحديثة أصبح لها بورت ثابت ومحدد وهو 2049.
- **المرونة عبر الشبكات:** لا يشترط أن تكون الأجهزة على نفس الشبكة أو نفس نظام التشغيل تماماً لمشاركة الملفات بسلاسة، as long as the two operating systems support the NFS protocol.
- **التكامل والأمان:** يدعم ميزات متقدمة في الإدارة والتحكم في الوصول للملفات بغض النظر عن الاختلافات بين الأجهزة.
- **استمرارية الوصول:** يساعدك في الحفاظ على حالة الاتصال واستعادة الوصول للبيانات، بحيث تستطيع جلب الـ data الخاصة بك بغض النظر عن الـ physical location (الموقع الفيزيائي).
- **إدارة التخزين (Storage):** يساعدك أيضاً في أن تقوم بـ add more storage (إضافة مساحة تخزين إضافية) للـ client الخاص بك بسهولة عبر الشبكة.

Setup and Configure NFS Mounts on Linux Server

- To setup NFS mounts, we'll be needing at least two Linux/Unix machines.
 - ✓ NFS Server
 - ✓ NFS Client

Installing NFS Server and NFS Client

- We need to install NFS packages on our NFS Server as well as on NFS Client machine.

```
[root@master ~]# yum install nfs-utils  
[root@master ~]# systemctl start nfs-server  
[root@master ~]# systemctl enable nfs-server  
[root@master ~]# systemctl stop firewalld
```

Setting Up the NFS Server

```
[root@master ~]# mkdir /data
[root@master ~]# vim /etc/exports
/data      192.168.1.0/24(ro)
/data      192.168.1.101(rw)
[root@master ~]# systemctl restart nfs-server    (terminates all open sessions)
[root@master ~]# exportfs -r                    (re-read the config file without termination of open sessions)
[root@master ~]# exportfs -f                    (force re-read the config file)
```


1. تجهيز البيئة وتثبيت الحزم

Environment & Installation

1

يتم تثبيت حزمة الـ nfs على كلا من الـ NFS Server والـ NFS Client:

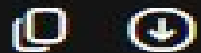
```
yum install nfs-utils
```

2. Server Configuration - إعدادات خادم ملفات المشاركة

Server Configuration

4

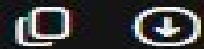
تفعيل وتشغيل خدمة الـ NFS:



Bash

```
systemctl start nfs-server  
systemctl enable nfs-server
```

- نطاق أوامر الخدمة: أوامر الـ start و enable والتحكم بالخدمة تخص السيرفر فقط وليست مطلوبة على جهاز الكلينت.
- التعامل مع الجدار الناري: إيقاف الـ firewalld عبر الـ `systemctl stop firewalld` هو إجراء مؤقت للتجربة لحين دراسة موضوع الـ Firewall بالتفصيل، والطريقة الصحيحة برمجياً وأمنياً هي إضافة خدمة الـ NFS داخل جدار الحماية (firewall-cmd).



Plaintext

```
/data    192.168.1.0/24(ro)
/data    192.168.1.101(rw)
```



Bash

```
systemctl restart nfs-server # (terminates all open sessions)
exportfs -r                  # (re-read the config file without termination of
exportfs -f                  # (force re-read the config file)
```

- ملف التهيئة الرئيسي (/etc/exports/) هو ملف ال configuration الرئيسي للـ NFS services، وهو المسؤول عن إضافة ال records الخاصة بال directory التي تريد مشاركتها وتحديد ال IP أو ال Network range لصلاحياتها. حيث يتم كتابة مسار المجلد والذي يليه ال IP (specific IP)، وبين قوسين تحط ال options بالانجليزي مثل (ro) read-only أو (read-write) write (rw)، أو استخدام ال star وهي ال asterisk (*) to allow for anyone.

- تأثير إعادة تشغيل الخدمة (Impact): استخدام أمر ال restart للخدمة قد يتسبب في عمل impact لو في مستخدمين فاتحين NFS sessions، ولذلك يُفضل استخدام البدائل الآمنة مثل exportfs -r أو exportfs -f.

Setting Up the NFS Client

```
[root@client ~]# showmount -e 192.168.1.100
[root@client ~]# mount 192.168.1.100:/data /root/data
[root@client ~]# df -hT          (nfs4)
[root@client ~]# mount -t nfs 192.168.1.100:/data /root/data
[root@client ~]# vim /etc/fstab
```

```
192.168.1.100:/data    /root/data    nfs    defaults    0    0
```

1. طرق الاستعلام عن المجلدات المشتركة (Query Exports)

Querying Shared Directories

على جهاز الكلاينت، لمعرفة المجلدات التي قام السيرفر بمشاركتها، لديك طريقتان:

- **الطريقة الأولى (مع الإصدارات القديمة مثل NFSv3):** باستخدام أمر الاستعلام المباشر:

```
showmount -e 192.168.1.100
```

- **الطريقة الثانية (مع الإصدارات الحديثة NFSv4):** بما أن NFSv4 لا يعتمد على `showmount`

بالشكل التقليدي، يمكنك عمل ماونت لجذر السيرفر (/) للاستكشاف فقط ومعرفة أسماء المجلدات المتاحة:

```
mount 192.168.1.100:/ /root/data
```

ثم الدخول وعرض محتوياتها عبر `ls /root/data`

(ملاحظة تفرق عن الماونت اليدوي: عمل ماونت للجذر / هدفه الاستكشاف ومعرفة المجلدات المتاحة فقط ولا يُخصص للعمل الفعلي، بينما استهداف مجلد بعينه يتيح لك الدخول والعمل الفعلي عليه).

إذا أردت العمل الفعلي على مجلد محدد وتعديل ملفاته:

```
mount 192.168.1.100:/data /root/data
```

أو بالشكل التفصيلي:

```
mount -t nfs 192.168.1.100:/data /root/data
```

- **التوضيح:** هنا أنت تستهدف مجلد معين بذاته (/data) لفتح قناة عمل مباشرة، وتستطيع الدخول إليه والتعامل مع محتوياته (قراءة، كتابة، تعديل، حذف) بناءً على الصلاحيات الممنوحة لك (read-only أو read/write).
- **للتأكد من نجاح العملية:**
`df -hT` (ويظهر نوع الملفات nfs4).

3. طريقة الماونت الدائم (Permanent Mount via /etc/fstab)

Permanent Mounting

لضمان بقاء المجلد معمولاً له ماونت بشكل دائم حتى بعد إعادة تشغيل الجهاز (Reboot):
فتح ملف التهيئة الرئيسي:

```
vim /etc/fstab
```

وإضافة هذا الريكورد في نهاية الملف:

```
data /root/data nfs defaults 0 0/:192.168.1.100
```

4. الطريقة الثالثة: الماونت عند الطلب (Mount on Demand / AutoFS)

Mount on Demand (AutoFS)

- تعتمد على خدمة ذكية تسمى AutoFS (أو Automount).
- الفكرة: المجلد لا يتم عمل ماونت له تلقائياً مع الإقلاع، ولكن فقط عندما يحاول المستخدم الدخول عليه أو طلبه (On Demand).
- وإذا انتهى الاستخدام ومَرَّ وقت محدد بدون أي نشاط (Timeout)، يقوم النظام بعمل unmount أوتوماتيكياً توفيراً لموارد النظام والشبكة.

1. تثبيت حزم الـ NFS

`dnf install nfs-utils` : يُستخدم لتثبيت حزمة أدوات الـ NFS على كل من السيرفر (`server`) والكلاينت (`client`) لتفعيل الدعم الكامل للخدمة.

2. تفعيل خدمة الـ NFS والتحقق منها

`systemctl enable --now nfs-server` : لتفعيل وتشغيل خدمة الـ NFS على السيرفر فوراً ومع إقلاع النظام.

`systemctl status nfs-server` : للتحقق من حالة الخدمة والتأكد من أنها تعمل (`active`).

3. فحص وإعداد الجدار الناري (Firewalld)

`systemctl status firewalld` : للتحقق من حالة جدار الحماية ومعرفة ما إذا كان يعمل (`active/running`).

`firewall-cmd --permanent --add-service=nfs` : لإضافة خدمة الـ NFS بشكل دائم إلى جدار الحماية للسماح بالاتصالات الواردة.

`firewall-cmd --reload` : لتطبيق وإعادة تحميل قواعد الجدار الناري الجديدة دون قطع الاتصالات الحالية.

`firewall-cmd --list-all` : لعرض جميع الخدمات والقواعد المفعلة للتأكد من ظهور `nfs` ضمن قائمة الـ `services` .

1. `mkdir /external` : لإنشاء المجلد الجديد على السيرفر والذي سيتم مشاركته عبر شبكة الـ NFS.

2. التعرف على بيانات الكلاينت أولاً

`ifconfig` : لمعرفة عنوان الآي بي الخاص بجهاز الكلاينت (192.168.2.132) لكي نكون على دراية به أثناء كتابة القواعد داخل الملف.

`hostname` : لمعرفة اسم الجهاز والدومين الخاص به (client.iti.com) لمطابقته بدقة في إعدادات السيرفر.

3. كتابة وصيغ الإعدادات داخل ملف التصدير

`vim /etc/exports` : لفتح وتعديل ملف التهيئة الرئيسي، حيث نكتب داخله الصيغ والأشكال المختلفة لتحديد الصلاحيات:

o `external 192.168.2.132(rw)/` : السماح لأي بي محدد بصلاحيات القراءة والكتابة.

o `external 192.168.2.0/24(ro)/` : السماح لنطاق شبكي كامل بصلاحيات القراءة فقط.

o `external *(rw)/` : استخدام النجمة (*) بمعنى `allow for anyone` للسماح لأي جهاز.

o `external client.iti.com(ro, sync)/` : تحديد اسم جهاز معين بالدومين الخاص به.

o `external *.iti.com(rw, sync, no_root_squash)/` : السماح لأي جهاز داخل الدومين بالكامل باستخدام النجمة.

o **ملحوظة هامة:** بعد تجربة واختبار هذه الصيغ، سنقوم بتنظيف الملف والكفاءة بالسطر الأول فقط (الخاص بالـ IP المطلوب) ومسح باقي الأسطر التجريبية.

1. تطبيق وتحديث التعديلات عبر أدوات `exportfs`
- `exportfs -r` : إعادة قراءة ملف التهيئة وتطبيق التعديلات دون قطع اتصالات العملاء الحاليين.
 - `exportfs -a` : تصدير أو إلغاء تصدير جميع الديركتوريات المتاحة دفعة واحدة.
 - `exportfs -ua` : إلغاء تصدير كل المجلدات المشاركة (`unexport all`).
 - `exportfs -v` : لعرض تفاصيل المجلدات المشاركة مع إظهار جميع الخيارات الافتراضية المفعلة.
 - `man 5 exports` : لفتح دليل الاستخدام والتعرف على تفاصيل كل خيار من مانوال النظام.

الأوبشن (Option)	المعنى والشرح بالعربي
<code>secure</code>	يفرض أن تكون الطلبات قادمة من بورت غير مميز (أقل من 1024)، وهو مفعّل افتراضياً لأسباب أمنية.
<code>rw</code>	السماح بعمليات القراءة والكتابة معاً على المجلد (افتراضياً ممنوع ويجب تحديده لتفعيل الكتابة).
<code>async</code>	السماح للسيرفر بالرد على العميل قبل كتابة البيانات فعلياً على القرص الصلب (يحسن الأداء لكنه يمثل خطراً لفقدان البيانات عند السقوط المفاجئ).
<code>sync</code>	لا يتم الرد على العميل إلا بعد التأكد من حفظ البيانات وكتابتها بالكامل على القرص الصلب (أكثر أماناً).
<code>no_wdelay</code>	يقلل التأخير المعتاد في دمج طلبات الكتابة المتشابهة لنفس الملف لزيادة الأداء عند كثرة الطلبات الصغيرة.
<code>nohide</code>	يمنع إخفاء نظام ملفات فرعي متداخل داخل مجلد مشترك، مما يتيح للعميل الوصول إليه مباشرة.

خاصية عدم تأخير الكتابة (no_wdelay)

- **الفكرة الدقيقة:** عندما يرسل العميل طلبات كتابة أو تعديل متتالية وسريعة على نفس الملف (أي كتل بيانات مرتبطة ومتتالية تخص نفس الملف الواحد، وليس بين ملفات مستقلة ومختلفة عن بعضها)، يقوم السيرفر في الوضع العادي (wdelay) بعمل تأخير بسيط جداً (أجزاء من الثانية) ليتحقق مما إذا كان هناك طلبات كتابة أخرى تابعة لنفس الملف في الطريق، ليقوم بدمجها معاً وكتابتها دفعة واحدة على الهارد تحسباً للأداء.
- **المثال مع التفعيل:** لو استخدمت no_wdelay ، فأنت تقول للسيرفر: "متستناش ولا جزء من الثانية، أي طلب كتابة أو تعديل يجيلك لنفس الملف نفذه واكتبه على الهارد فوراً بدون أي دمج أو تأخير زمني".

خاصية إظهار الملفات المتداخلة (nohide)

- **الفكرة:** تخيل أن لديك على السيرفر مجلد رئيسي اسمه data/ ، ومثبت أو موصل بداخله هارد أو بارتيشن فرعي آخر اسمه data/docs/ (أي نظام ملفات داخل نظام ملفات).
- **الوضع الافتراضي (بدون nohide):** الكلاينت عندما يقوم بعمل ماونت للمجلد الرئيسي data/ ويدخل بداخله إلى data/docs/ سيجده فارغاً تماماً، وسيضطر لعمل ماونت منفصل ومستقل لمجلد ال docs لكي يرى محتوياته.
- **المثال مع nohide :** عند تفعيل هذا الخيار، يتم إلغاء هذا الإخفاء؛ وبمجرد أن يقوم الكلاينت بعمل ماونت للمجلد الرئيسي data/ ، يستطيع الدخول مباشرة إلى المجلد الفرعي data/docs/ ورؤية كل ملفاته الداخلية دون الحاجة لعمل ماونت ثاني منفصل.

1.

الاستعلام عن المجلدات المشتركة (On Client)

`showmount -e 192.168.2.133` : أمر الاستعلام عن المجلدات المشاركة على السيرفر (يتم تنفيذه على الكلاينت)، ولكن قد تظهر أخطاء مثل `RPC: Unable to receive` إذا لم تكن خدمات ال `RPC` وال `Firewall` مجهزة بشكل صحيح.

2.

إصلاح أخطاء الاتصال بفتح خدمات ال `RPC` في الفايروال (على السيرفر)

`firewall-cmd --permanent --add-service=rpc-bind` : للسماح بخدمة ربط ال `RPC` بصفة دائمة في الجدار الناري لتفادي أخطاء الاتصال.

`firewall-cmd --permanent --add-service=mountd` : للسماح بخدمة إدارة نقاط الماونت `mountd` في الفايروال.

`firewall-cmd --reload` : لتطبيق وتحديث قواعد الجدار الناري فوراً دون قطع الخدمات.

`firewall-cmd --list-all` : لعرض الخدمات المفعلة للتأكد من وجود `nfs` و `rpc-bind` و `mountd`.

o

ملحوظة هامة للامتحان: إذا واجهتك أخطاء اتصال أو استعلام عن ال `NFS` أثناء الامتحان، قم فوراً بفتح وتفعيل الخدمات الثلاثة الأساسية في الفايروال وهي (`nfs` , `rpc-bind` , `mountd`) لضمان نجاح الاتصال والاستعلام تماماً.

1.

تنفيذ أمر الاستعلام الناجح (On Client)

`showmount -e 192.168.2.133` : بعد ضبط الفايروال، يعرض هذا الأمر بنجاح قائمة المجلدات المشاركة مثل `/external`.

2.

عمل الماونت اليدوي والاستكشاف (On Client)

`mount 192.168.2.133:/ /mnt` : طريقة الماونت لجذر السيرفر (/) بغرض الاستكشاف ورؤية المجلدات المتاحة.

`df -h` : للتحقق من نجاح عملية ربط نظام الملفات الشبكي وظهوره في الجدول.

`cd /mnt` ثم `ls` : الدخول للمجلد وعرض محتوياته للاستكشاف.

`umount /mnt` : لفك الماونت وإنهاء الاتصال بالمجلد المؤقت.

○ إنشاء مجلد نقطة الربط (Mount Point):

○ `mkdir /mnt/external` (On Client): يُستخدم لإنشاء مجلد جديد محلي على جهاز الكلاينت، والذي سيتم ربط المجلد الشبكي عليه ليكون بمثابة البوابة لعرض الملفات.

○ عمل الماونت المؤقت:

○ `/mount -t nfs 192.168.2.133:/external /mnt/external` (On Client): أمر ربط مؤقت يقوم بوصل المجلد المشارك القادم من السيرفر (الذي يحمل الآبي 192.168.2.133) داخل المجلد المحلي `/mnt/external/` باستخدام بروتوكول `nfs`.

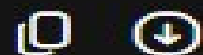
○ التحقق من نجاح الربط المؤقت:

○ `df -hT` (On Client): لعرض أنظمة الملفات النشطة وتأكد ظهور السيرفر ضمن الجدول مع نوع نظام الملفات (مثل `nfs4`) ومساحته وحجم الاستخدام.

○ جعل الماونت دائماً وثابتاً بعد إعادة التشغيل (`etc/fstab/`):

○ `vi /etc/fstab` (On Client): فتح ملف النظام المسؤول عن تعريف أقراص التخزين وربطها تلقائياً عند الإقلاع.

○ السطر المضاف في نهاية الملف:



Plaintext

```
192.168.2.133:/external /mnt/external nfs4 rw, sync, soft 0 0
```


(تفسير السطر: عنوان الآبي والمجلد المشترك / مكان الربط محلياً / نوع الملفات `nfs4` / الخصائص المستخدمة وهي القراءة والكتابة `rw` ، والمزامنة الآمنة `sync` ، ووضع التعامل `soft` لعدم تعليق الجهاز لو السيرفر فصل / والإعدادات الأخيرة `0 0` الخاصة بفحص الأقراص والدامب).

○ اختبار الإعدادات الجديدة وتحديث النظام:

○ `umount /mnt/external` (On Client): فك الماونت المؤقت القديم تمهيداً لاختبار الإعدادات الجديدة من ملف `fstab`.

○ `mount -a` (On Client): أمر يعيد قراءة ملف `fstab` وتطبيق كل الأسطر الموجودة بداخله تلقائياً (بما فيها ماونت الـ NFS الذي أضفناه للتو).

○ ظهور تنبيه (Hint) يطلب تحديث مدير النظام بسبب تعديل ملف `fstab`.

○ `systemctl daemon-reload` (On Client): أمر لتحديث وتنشيط مدير النظام `systemd` ليتعرف على التعديلات الجديدة ويخفي رسالة التحذير.

○ التحقق النهائي والتعامل مع الملفات:

○ `df -hT` (On Client): تنفيذ أمر العرض مرة أخرى للتأكد من نجاح الماونت الدائم وثباته، وبمجرد ظهوره هكذا تصبح قادراً على الدخول للمجلد `/mnt/external/` وإنشاء وتعديل وحفظ أي ملفات ترغب فيها بكل سهولة.

- **مفهوم الـ Root Squashing:** في بروتوكول NFS، لأسباب أمنية صارمة، يتم افتراض أن مستخدم الـ `root` المحلي على جهاز الكلاينت ليس بالضرورة أن يكون موثقاً على السيرفر، لذلك يتم تطبيق ميزة تسمى `root_squashing` (وهي مفعلة افتراضياً)، والتي تحول صلاحيات مستخدم الـ `root` القادم من الكلاينت إلى مستخدم مجهول وضعيف الصلاحيات على السيرفر يُسمى `nfsnobody` (أو `nobody`).

- **تعارض صلاحيات الـ NFS مع الصلاحيات القياسية (Standard Permissions):**

- الإعدادات في ملف `etc/exports/` (مثل منح `rw`) تحدد فقط الحد الأقصى المسموح به عبر الشبكة، ولكنها لا تلغي أبداً صلاحيات نظام الملفات المحلي (`chmod` و `chown`).
- بما أن مستخدم الـ `root` تحول إلى `nobody` على السيرفر، فإنه في نظر النظام أصبح مستخدماً عادياً (Others) وليس المالك الأصلي (`owner`).
- لذلك، إذا كان المجلد على السيرفر يمنح المالك صلاحيات معينة بينما لا يمنح مستخدم الـ `nobody` (أو مجموعة Others) صلاحية الكتابة (`w`)، فسيتم منع الكتابة تماماً بغض النظر عن وجود خيار `rw` في ملف الـ `exports`، لأن الصلاحيات المحلية الأضعف هي التي تفرض سيطرتها وتمنع العملية.

Client Actions (تحركات الكلاينت)

- `/cd /mnt/external` : الانتقال إلى مسار المجلد المشترك على الكلاينت.
- `touch file1 file2` : محاولة إنشاء ملفات بحساب الـ `root` والتي فشلت وظهرت رسالة `Permission denied` بسبب تطبيق الـ `Root Squashing` وتحويل الـ `root` إلى مستخدم `.nobody`.
- `su karim` : التبديل إلى المستخدم العادي `.karim`.
- `touch file3 file4` : محاولة إنشاء الملفات بواسطة المستخدم `.karim`.

Server Actions (إجراءات السيرفر لتعديل الصلاحيات)

- `cat /etc/exports` : عرض ملف إعدادات الـ NFS للتأكد من تفعيل خيار الـ `.rw`.
- `ls -ld /external` : فحص الصلاحيات القياسية للمجلد (ظهرت `drwxr-xr-x` بحيث لا تملك فئة الـ `others` صلاحية الكتابة).
- `chmod o+w /external` : تعديل الصلاحيات ومنح صلاحية الكتابة (Write) للـ `others` ليتمكن مستخدم الـ `nobody` والجميع من الإنشاء والتعديل.
- `ls -ld /external` : التحقق من تحديث الصلاحيات للمجلد على السيرفر.

Client Actions (العودة للكلاينت بعد التعديل)

- `exit` : الخروج من حساب `karim` والعودة لحساب الـ `root` على الكلاينت.
- `touch file1 file2` : إعادة محاولة إنشاء الملفات بحساب الـ `root` والتي نجحت هذه المرة بعد تعديل الصلاحيات على السيرفر.

Server Actions (الاستعراض والتحقق النهائي على السيرفر)

- o `/cd /external` : الانتقال إلى المجلد المحلي على السيرفر.
- o `ls -l` : استعراض محتويات المجلد؛ حيث تظهر `file1` و `file2` مملوكتين للمستخدم `nobody` (نتيجة ال `root squashing`)، بينما `file3` و `file4` مملوكتان للمستخدم `karim`.
- o `grep nobody /etc/passwd` : لعرض وتأكيد تفاصيل حساب المستخدم `nobody` على نظام السيرفر.

Server Actions (تعديل إعدادات ال NFS وتطبيق خيار `no_root_squash`)

- o `exportfs -v`
- o عرض حالة ال `exports` الحالية والخيارات المفعلة قبل التعديل للتأكد من الجدول النشط.
- o `vi /etc/exports`
- o تعديل ملف الإعدادات وإضافة خيار `no_root_squash` للسماح لمستخدم ال `root` على الكلاينت بالاحتفاظ بصلاحياته الحقيقية على السيرفر (وهو خيار غير مستحب أمنياً `not recommended for security reasons` لأنه يمنح صلاحيات مطلقة لمدير الكلاينت على ملفات السيرفر).
- o `exportfs -r`
- o إعادة تحميل وتطبيق إعدادات ال NFS فوراً دون الحاجة لإيقاف الخدمة.
- o `exportfs -v`
- o التحقق من تطبيق التعديلات وظهور خيار `no_root_squash` الجديد، وتأكيد أن هذه هي الخيارات الفعلية والمطبقة حالياً على المجلد المشترك.

Client Actions (الكلاينت بعد تفعيل `no_root_squash`)

`touch file5 file6` ○

○ محاولة إنشاء ملفات بواسطة الـ `root` على الكلاينت وقد نجحت تماماً بعد تفعيل الخيار.

Server Actions (الاستعراض والتحقق النهائي على السيرفر)

`ls -l` ○

○ استعراض الملفات؛ وتلاحظ هنا أن الملفات التي تم إنشاؤها بواسطة الـ `root` على الكلاينت أصبحت تظهر باسم `root` كمالك ومسؤول (Owner & Group) وليست `nobody` ، لأن خيار `no_root_squash` ألغى عملية التحويل.

`ls -ld /external` ○

○ فحص خصائص المجلد الرئيسي.

`chmod o-w /external` ○

○ تعديل صلاحيات المجلد وإزالة الكتابة عن الـ `others` .

`ls -ld /external` ○

○ التحقق من تحديث الصلاحيات.

`ls -ltr` ○

○ عرض محتويات المجلد مرتبة زمنياً.

ملاحظات هامة:

○ خيار `all_squash` :

يقوم بعمل Mapping لجميع المستخدمين العاديين (Regular users) ليتم التعامل معهم كمستخدم موحد (مثل `nobody`) بغض النظر عن أسمائهم الحقيقية على الكلاينت.

○ خيار `no_all_squash` :

يعني ألا يتم عمل Mapping للـ Regular users، حيث يكون بأسمائهم العادية وهوياتهم الحقيقية على الكلاينت، ويأخذون صلاحيات الـ `others` أو الصلاحيات المتاحة لهم على السيرفر دون دمجهم.

○ مشاركة عدة مسارات في سطر واحد:

يتيح لك نظام الـ NFS إمكانية تصدير ومشاركة أكثر من مجلد في نفس السطر داخل ملف الإعدادات بمرونة، مثل المثال الموضح في دليل الاستخدام:

```
foo 2001:db8:9:e54::/64(rw) 192.0.2.0/24(rw)/
```

○ أمر `exportfs -v` :

يظهر لك جميع الـ exports والـ options والـ clients الموجودة والنشطة حالياً على السيرفر.

Automounting Network-Attached Storage

Mounting NFS Shares with the Automounter

- The automounter is a service (**autofs**) that automatically mounts NFS shares "on-demand," and will automatically unmount NFS shares when they are no longer being used.

Automounter Benefits

- Users do not need to have root privileges to run the **mount** and **umount** commands.
- NFS shares configured in the automounter are available to all users on the machine, subject to access permissions.
- NFS shares are not permanently connected like entries in **/etc/fstab**, freeing network and system resources.
- The automounter is configured on the client side; no server-side configuration is required.

- في الطريقتين الأوليين التقليديتين (ال Manual mount المباشر أو ال Persistent mount عبر ملف `etc/fstab/`), كان مستخدم ال `root` هو الوحيد الحصري الذي يمتلك صلاحية تنفيذ وتطبيق أوامر ال `mount` وال `umount`.

Mounting NFS Shares with the Automounter:

- خدمة ال `autofs`: هي خدمة تعمل على ربط مشاركات ال NFS بشكل تلقائي "عند الطلب" (On-demand), وتتولى إلغاء الربط (Unmount) تلقائياً عندما تتوقف عن الاستخدام ولا تكون مطلوبة.

Automounter Benefits (مميزات ال Automounter):

- **صلاحيات المستخدمين:** لا يحتاج المستخدمون العاديون إلى امتلاك صلاحيات ال `root` لتشغيل أوامر ال `mount` وال `umount`.
- **الإتاحة العامة:** تكون مشاركات ال NFS المُعرفة في ال automounter متاحة لجميع المستخدمين على الجهاز، بشرط خضوعهم لصلاحيات الوصول المعتادة.
- **توفير الموارد:** لا تبقى مشاركات ال NFS متصلة بشكل دائم على عكس الإدخالات في ملف `etc/fstab/`، مما يساعد على تحرير وتوفير موارد الشبكة والنظام.
- **التكوين المحلي:** يتم إعداد ال automounter بالكامل من جهة العميل (Client-side), ولا يتطلب أي تعديل أو إعدادات نهائياً من جهة السيرفر.

المقصود بالإتاحة العامة أن مشاركات الـ NFS المُعرّفة عبر الـ automounter تصبح متاحة للاستخدام لجميع المستخدمين المتواجدين على جهاز الكلاينت بشكل عام، وليست حكراً على حساب الـ root وحده كما كان في الطرق التقليدية.

○ الطرق التقليدية (Traditional Mounts / etc/fstab/):

- كان مستخدم الـ root على الكلاينت هو الوحيد الحصري الذي يمتلك الصلاحية لتنفيذ أمر الـ mount المبدئي لربط المجلد.
- بمجرد أن يقوم الـ root بعمل الـ mount، يستطيع أي مستخدم عادي على الكلاينت (مثل karim) الدخول والوصول للملفات طبيعياً، بشرط أن تسمح له الصلاحيات القياسية بذلك.

○ خدمة الـ Automounter (autofs):

- يلغي الاعتماد المطلق على الـ root لعمل الـ mount اليدوي، حيث لا يحتاج المستخدمون العاديون إلى امتلاك صلاحيات الـ root لتشغيل أوامر الـ mount والـ umount.
- يتيح المشاركة لجميع مستخدمي الكلاينت بشكل تلقائي "عند الطلب" (On-demand).

○ مثال توضيحي لفكرة الإتاحة والوصول على الكلاينت:

- تخيل أن المستخدم العادي karim يريد الدخول على المجلد المشترك من جهازه.
- في الطريقة التقليدية: إذا كان المجلد غير مربوط، فلن يتمكن karim من الوصول إليه مطلقاً حتى يقوم الـ root بتنفيذ أمر الـ mount يدوياً أولاً.
- مع الـ Automounter: بمجرد أن يحاول karim الدخول إلى المسار، تقوم خدمة الـ autofs بربط المجلد تلقائياً في الخلفية وتفتح له الطريق فوراً، مع بقاء استخدامه خاضعاً دائماً لصلاحيات نظام الملفات القياسية.

Steps for Automounting NAS Using Autofs

1. Install the autofs package.

- Ensure Autofs is installed on your RHEL 9 system

```
[user@host ~]# dnf install autofs
```

2. Add a master map file to /etc/auto.master.d. This file identifies the base directory used for mount points and identifies the mapping file used for creating the automounts.

```
[user@host ~]# vim /etc/auto.master.d/demo.autofs
```

- Add the master map entry, in this case, for indirectly mapped mounts:

```
/shares /etc/auto.demo
```

○ `dnf install autofs` : التأكد من تثبيت حزمة الـ `autofs` على نظام RHEL 9.

○ إنشاء ملف الـ Master Map (Step 2):

○ يتم إضافة ملف الـ master map داخل المجلد المخصص `/etc/auto.master.d/`.

○ ملحوظة هامة على الاسم وامتداد الملف: يجب أن يكون الامتداد الخاص بملف الـ master mapping هو `autofs` حصرياً ، بينما الاسم الذي يسبقه (مثل `demo`) ليس إلزامياً ويمكن تغيير هذا الاسم بأي اسم آخر مناسب ومخصص (مثل `demo.autofs`).

○ أمر فتح الملف والتعديل عليه: `vim /etc/auto.master.d/demo.autofs`.

○ إضافة الـ Record أو الـ Entry داخل الملف:

○ يتم إضافة سطر التوجيه الخاص بالربط (للحالات غير المباشرة Indirect)، مثل:
`shares /etc/auto.demo/`

○ الجزء `shares/` يمثل الـ base directory الأساسي لنقاط الربط.

○ الجزء `/etc/auto.demo/` يمثل ملف الـ mapping المحدد في الـ master map، مع التنويه بأن اسم `demo` ليس إلزامياً ويمكن تغيير الاسم بأي اسم آخر مناسب.

○ يختلف الـ structure الخاص بهذا الـ entry بوضوح بناءً على نوع الربط، سواء كان من نوع الـ Direct أو الـ Indirect.

Steps for Automounting NAS Using Autofs

3. Create the mapping files. Each mapping file identifies the mount point, mount options, and source location to mount for a set of automounts.

```
[user@host ~]# vim /etc/auto.demo
```

- The mapping file-naming convention is /etc/auto.name, where name reflects the content of the map

```
work -rw,sync serverb:/shares/work
```

4. Start and enable the automounter service.

- Use systemctl to start and enable the autofs service.

```
[user@host ~]$ sudo systemctl enable --now autofs
```

○ تكوين ملف ال Map الخاص بال Indirect Mapping:

○ بعد إنشاء ملف ال master map (`etc/auto.master.d/demo.autofs/`)، يتم إنشاء وفتح ملف ال mapping نفسه (مثل `etc/auto.demo/`) لإضافة ال Entry أو ال Record الخاص بالمجلدات.

○ هيكل ال entry داخل هذا الملف يتكون من:

1. اسم ال Mount Point الوهمي أو الفرعي (مثل `external` أو أي اسم تختاره لتظهر بداخله المشاركة).

2. خيارات الربط بين قوسين (إن وجدت، مثل `rw,soft-`).

3. عنوان السيرفر ومسار المجلد المشترك عليه (مثلاً: `server:/external` أو ال IP الخاص بالسيرفر مع المسار المراد ربطه).

○ خطوة التفعيل والتجربة (On-demand Mount):

○ بعد حفظ الملفات، تقم بإعادة تشغيل أو تفعيل خدمة ال `autofs` على الكلاينت لتطبيق الإعدادات.

○ ملحوظة تجريبية هامة: هذه تعتبر طريقة ال Indirect Mapping. بمجرد تنفيذ أمر الانتقال `cd /shares` ثم الدخول إلى ال mount point الفرعي (مثل `cd external`)، ستقوم خدمة ال `autofs` تلقائياً بعمل ال mount في الخلفية وحدها دون أي تدخل يدوي منك، وسيعمل معك المجلد فوراً وتستطيع استعراض محتوياته بسلاسة.

Direct Maps

- Direct maps are used to map an NFS share to an existing absolute path mount point.
- To use directly mapped mount points, the master map file might appear as follows:

```
/- /etc/auto.direct
```

- All direct map entries use `/-` as the base directory. In this case, the mapping file that contains the mount details is **`/etc/auto.direct`**.
- The content for the `/etc/auto.direct` file might appear as follows:

```
/mnt/docs -rw,sync serverb:/shares/docs
```

- The mount point (or key) is always an absolute path. The rest of the mapping file uses the same structure.
- In this example the **`/mnt`** directory exists, and it is not managed by **autofs**. The full directory **`/mnt/docs`** will be created and removed automatically by the **autofs** service.

مفهوم ال Direct Maps:

تُستخدم لربط مشاركة NFS بنقطة ربط ذات مسار مطلق (Absolute path mount point) معروف وثابت على الكلاينت، وليست تحت مجلد رئيسي موحد كما في ال Indirect Maps.

1. إدخال ملف ال Master Map:

- الأمر: `etc/auto.direct/ -/`
- الشرح: الرمز `-/` يُشير إلى أن المسارات داخل ملف ال mapping التابع له ستكون مسارات مطلقة بالكامل (Absolute paths). و `etc/auto.direct/` هو ملف ال mapping الذي يحتوي على تفاصيل هذه الربطات المباشرة.

2. محتوى ملف ال Mapping (`etc/auto.direct/`):

- الأمر: `mnt/docs -rw, sync serverb:/shares/docs/`
- تفاصيل أجزاء السطر:
- `mnt/docs/` : مسار ال Mount Point (أو المفتاح) وهو مسار مطلق كامل يتم تعريفه محلياً.
- `-rw, sync` : خيارات الربط (Options) مثل السماح بالقراءة والكتابة (`rw`) والمزامنة (`sync`).
- `serverb:/shares/docs` : عنوان السيرفر (`serverb`) متبوعاً بالمجلد المشترك المصدر من السيرفر (`shares/docs/`).

3. آلية عمل ال Direct Maps على الكلاينت:

- المجلد الأساسي الحاضن (`mnt/`) يجب أن يكون موجوداً محلياً على الكلاينت، ولكنه غير مُدار بواسطة خدمة ال `autofs`.
- المجلد النهائي (`mnt/docs/`) لا يتم إنشاؤه مسبقاً بشكل دائم، بل تقوم خدمة ال `autofs` بإنشائه وإزالته تلقائياً عند الطلب (On-demand) بمجرد محاولة الوصول إليه.

Indirect Wildcard Maps

- When an NFS server exports multiple subdirectories within a directory, then the automounter can be configured to access any one of those subdirectories using a single mapping entry.
- Continuing the previous example, if `serverb:/shares` exports two or more subdirectories and they are accessible using the same mount options, then the content for the **/etc/auto.demo** file might appear as follows:

```
* -rw,sync serverb:/shares/&
```

- The mount point (or key) is an asterisk character (*), and the subdirectory on the source location is an ampersand character (&). Everything else in the entry is the same.
- When a user attempts to access **/shares/work**, the key * (which is **work** in this example) replaces the ampersand in the source location and `serverb:/shares/` is mounted. As with the indirect example, the **work** directory is created and removed automatically by autofs.

مفهوم ال Indirect Wildcard Maps:

تُستخدم هذه الطريقة عندما يقوم السيرفر بتصدير عدة مجلدات فرعية (subdirectories) داخل مجلد رئيسي واحد، حيث يتيح لك ال automounter الوصول إلى أي واحد من هذه المجلدات الفرعية باستخدام سطر (Entry) واحد فقط في ملف ال mapping بدلاً من تخصيص سطر منفصل لكل مجلد.

شكل ال Record أو ال Entry داخل ملف ال Mapping (مثل `etc/auto.d/demo/`):

```
&/rw, sync serverb:/shares- *
```

تفاصيل مكونات هذا السطر:

- النجمة (*): تُمثل ال Mount Point (أو المفتاح) وتعمل ك Wildcard لتلتقط أي اسم مجلد فرعي يحاول المستخدم الوصول إليه على ال client.

- `rw, sync-` : خيارات الربط (Options) المطبقة بشكل موحد على جميع المجلدات.

- `&/serverb:/shares` : مسار السيرفر، حيث يتم استخدام علامة ال & (Ampersand) كمتغير يتم استبداله تلقائياً بنفس الاسم الذي طلبه المستخدم.

كيف تعمل الآلية على ال client؟

- عندما يحاول مستخدم على ال client الدخول إلى مسار مثل `shares/work/`:

- تلتقط النجمة (*) الكلمة المطلوبة وهي (work).

- تقوم خدمة ال autofs بتعويض علامة ال & في مسار السيرفر بكلمة work ، ليصبح الاتجاه الفعلي نحو `serverb:/shares/work`.

- يتم إنشاء المجلد المحلي وربطه تلقائياً (On-demand)، ويتم إزالته تلقائياً عند انتهاء الاستخدام بواسطة ال autofs.

بالضبط كدة، كل إعدادات ال `autofs` ، وخدمة الأوتوماونتر، وملفات ال Master Map وال Mapping Files تتم بالكامل على جهاز ال Client وليس على السيرفر.

- دور السيرفر: يقتصر فقط على تجهيز ومشاركة المجلدات وتصديرها عبر ملف `etc/exports/` (مع ضبط صلاحيات ال `root_squash` ونحوها).
- دور ال Client: هو الذي يحتاج إلى تثبيت حزمة ال `autofs` ، وإنشاء مجلدات ال Master Map وال Mapping Files لديه، لكي يعلم متى وكيف يطلب المجلد ويربطه تلقائياً "عند الطلب" (On-demand).

خطوات إعداد وتنفيذ ال Direct Maps باستخدام خدمة ال `autofs` على ال client خطوة بخطوة:

○ 1. تثبيت حزمة ال `autofs` على ال client

- الأمر: `dnf install autofs`
- الشرح: يتم تثبيت حزمة الأوتوماونتر الأساسية المطلوبة لإدارة عملية ال mounting التلقائي على النظام.
- كيفية التحقق: التأكد من انتهاء التثبيت بنجاح وظهور رسالة "Complete!" أو نجاح المعاملة (Transaction summary) دون أخطاء.

○ 2. الانتقال إلى مجلد إعدادات ال Master Map

- الأمر: `/cd /etc/auto.master.d`
- الشرح: الانتقال إلى المجلد المخصص لوضع ملفات ال master mapping الخاصة بال client.
- كيفية التحقق: تنفيذ أمر `pwd` للتأكد من أنك تواجدت داخل المسار الصحيح `./etc/auto.master.d/`

3. إنشاء ملف ال Master Map الخاص بال Direct Mapping

الأمر: `vi direct.autofs`

الشرح: إنشاء ملف جديد بالامتداد الإلزامي `autofs.` (مثل `direct.autofs`) لتسجيل توجيه ال Direct Maps بداخله.

المحتوى المضاف داخل الملف:

```
etc/auto.direct/ -/
```

توضيح السطر: الرمز `-/` يعني أننا نتعامل مع مسارات مطلقة (Absolute paths)، وأن تفاصيل الربط ستكون مسجلة في ملف ال mapping الذي يليه وهو `etc/auto.direct/`.

كيفية التحقق: حفظ الملف والخروج منه عبر كتابة `wq`، ثم التأكد من وجوده داخل المجلد بأمر `ls -l`.

4. إنشاء وضبط ملف ال Mapping الفعلي (`etc/auto.direct/`)

الأمر: `vim /etc/auto.direct`

الشرح: فتح ملف ال mapping لتحديد تفاصيل ومسارات الربط المطلقة لل NFS share القادم من ال server.

المحتوى المضاف داخل الملف:

```
external -rw,sync,fstype=nfs4 192.168.2.133:/shares/direct/external/
```

توضيح أجزاء السطر:

`external/` : ال Absolute mount path (نقطة الربط المباشرة والمطلقة على ال client).


```
systemctl enable --now autofs
```

```
df -h
```

○ تجربة الاستخدام والتفعيل التلقائي:

○ الأمر: `echo testing-direct > /external/testing.txt`

○ الشرح: محاولة كتابة نص داخل مسار `external/testing.txt/` . بمجرد تنفيذ هذا الأمر، تكتشف خدمة الـ `autofs` طلب الوصول للمسار وتتدخل فوراً في الخلفية لربط الـ NFS share تلقائياً.

○ التحقق النهائي بعد الاستخدام:

○ الأمر: `df -h`

○ الشرح: عند إعادة تنفيذ أمر استعلام أنظمة الملفات بعد محاولة الكتابة، ستجد أن مسار السيرفر (`shares/direct/external/:192.168.2.133`) قد ظهر وأصبح مربوطاً بنجاح عند نقطة الربط `external/` .

○ ملحوظة هامة (Dynamic Umount):

○ إذا توقفت عن استخدام مشاركة الـ NFS ولم يتم إجراء أي عمليات أو وصول للمسار لفترة زمنية محددة (افتراضياً 5 دقائق / 300 ثانية)، فإن خدمة الـ `autofs` تقوم تلقائياً بعمل `umount` ديناميكي للمشاركة في الخلفية لتوفير الموارد، وستعود للعمل وربط نفسها تلقائياً فور محاولة الدخول إليها أو استخدامها مرة أخرى.



Bash

```
/- /etc/auto.direct --timeout=5
```

تم إضافة خيار `timeout=5--` لضبط خدمة الأوتو بحيث تقوم بفك المونت التلقائي بعد مرور خمس ثواني فقط من عدم استخدام المجلد أو الـ `shared directory`.

```
systemctl restart autofs
```



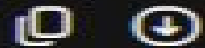
Bash

```
echo testing-direct > /external/testing.txt
df -h
sleep 5
df -h
```

○ شرح الاختبار والتنفيذ:

- كتابة نص داخل ملف في مسار `external/testing.txt/` لتفعيل الـ `mount` مؤقتاً عند الطلب.
- تنفيذ أمر `df -h` لعرض أنظمة الملفات والتأكد من ظهور ربط الـ `NFS share`.
- الانتظار لمدة 5 ثواني (`sleep 5`) وفقاً للقيمة المحددة في الـ `timeout`.
- إعادة تنفيذ أمر `df -h` للتحقق من إتمام الـ `umount` التلقائي للمجلد بعد انتهاء مدة الخمول.

1. الإعدادات على السيرفر (Server Side)



Bash

```
mkdir /shares/indirect  
cd /shares/indirect  
mkdir central west east  
vim /etc/exports
```

إضافة السطر داخل الملف:

```
shares/indirect 192.168.2.132(rw, sync, no_root_squash) /
```

```
exportfs -r
```

```
exportfs
```

2. إعدادات على ال Client (Client Side)

(إنشاء ال Master Mapping الخاص بال Indirect) `vim /etc/auto.master.d/indirect.autofs`

إضافة مسار ال Base Directory وربطه بالملف ال Mapping:

```
internal /etc/auto.indirect/
```

- ملحوظة هامة: مسار `internal/` هو ال Base Directory. بحيث إنه عندما تقوم بالوصول إلى أي subdirectory، سيتم عمل mount له تلقائياً تحت مسار `internal/` مباشرة، مع قيام خدمة ال autofs بفك ال mount تلقائياً في حال الخمول لتوفير الموارد.

إعداد ال Wildcard داخل ملف ال Indirect Mapping

```
vim /etc/auto.indirect
```



Plaintext

```
* -rw,sync,fstype=nfs4 192.168.2.133:/shares/indirect/&
```

- توضيح الإعداد: استخدام النجمة (*) يعني أي مجلد فرعي يتم طلبه تحت مسار ال Base Directory. وعلامة العطف (&) تعوض تلقائياً بنفس اسم المجلد المطلوب، بحيث يتم ربطه ديناميكياً بالمجلد المقابل له على السيرفر تحت مسار `/shares/indirect/`.

```
systemctl restart autofs
```

اختيار ال Wildcard Automounting على الكلاينت

تنفيذ اختبارات الوصول المباشر للمجلدات الفرعية (`central` , `east` , `west`):



Bash

```
echo testing-1 > /internal/west/testing-1.txt
echo testing-2 > /internal/east/testing-2.txt
echo testing-3 > /internal/central/testing-3.txt
```

التحقق من نجاح عملية ال mount التلقائي باستخدام أمر `df -h`:

إضافة مجلد جديد على السيرفر والاختبار الديناميكي

إنشاء مجلد `north` جديد على السيرفر:



Bash

```
mkdir north
```

الوصول إليه فوراً من الكلاينت واختباره دون الحاجة لتعديل أي ملفات إعدادات بفضل خاصية ال Wildcard:



Bash

```
echo testing-4 > /internal/north/testing-4.txt  
df -h
```

Summary

In this chapter, you learned how to:

- Mount and unmount an NFS export from the command line.
- Configure an NFS export to automatically mount at startup.
- Configure the automounter with direct and indirect maps, and describe their differences.

